

SKILL5.6.7.8

5. To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

1. To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

INPUT:

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    printf("\nOriginal Input:\n%s\n", input);

    printf("\nParsing quotes...\n");

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\'' ) {
            printf("Single quote found at position %d\n", i);
        }
        else if (input[i] == '"' ) {
            printf("Double quote found at position %d\n", i);
        }
    }

    return 0;
}
```

OUTPUT:

```
sushmitha@LAPTOP-B907B06L:~$ nano skill5.c
sushmitha@LAPTOP-B907B06L:~$ gcc skill5.c -o skill5
sushmitha@LAPTOP-B907B06L:~$ ./skill5
Enter command: echo 'skill5'

Original Input:
echo 'skill5'

Parsing quotes...
Single quote found at position 5
Single quote found at position 12
```

6. To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

2.

INPUT:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <stdlib.h>

int main() {
    char command[100];

    printf("Enter command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        char *args[20];
        int i = 0;

        char *token = strtok(command, " ");

        while (token != NULL && i < 19) {
            args[i++] = token;
            token = strtok(NULL, " ");
        }

        args[i] = NULL;

        execvp(args[0], args);

        perror("execvp failed");
    }
}
```

```

        execvp(args[0], args);

        perror("execvp failed");
        exit(1);
    }
    else {
        wait(NULL);
        printf("Command execution completed.\n");
    }

    return 0;
}

```

OUTPUT:

```

sushmitha@LAPTOP-B907B06L:~$ nano skill6.c
sushmitha@LAPTOP-B907B06L:~$ gcc skill6.c -o skill6
sushmitha@LAPTOP-B907B06L:~$ ./skill6
Enter command: ls
Screenshots  fifo_create  fifo_write.c  priority.c  sigchild  sigpipe.c  simplecommandloop  skill3a.c  skill6  sockets.c
client       fifo_create.c  myfifo       roundrobin  sigchild.c  sigsegv  simplecommandloop.c  skill3b  skill6.c  username
client.c     fifo_read     practical5b   roundrobin.c  signal     sigsegv.c  sjf_NONpreemptive  skill3b.c  socket  username.c
fcfs        fifo_read.c   practical5b.c  server.c     signal.c    sigterm   sjf_NONpreemptive.c  skill5  socket.c
fcfs.c      fifo_write    priority      server_side  sigpipe     sigterm.c  skill3a           skill5.c  sockets
Command execution completed.

```

7. Process Synchronization and Child Process Monitoring Using waitpid().

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

INPUT:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char *path = getenv("PATH");

    if (path == NULL) {
        printf("PATH variable not found.\n");
        return 1;
    }

    printf("PATH Variable:\n%s\n\n", path);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("Child Process Started\n");
        printf("Child PID: %d\n", getpid());

        // Execute ls
        execlp("ls", "ls", NULL);

        // Runs only if exec fails
        perror("exec failed");
        exit(1);
    }
    else {
        // Parent process
        int status;
```

```
// Parent process
int status;

printf("Parent waiting for child...\n");

waitpid(pid, &status, 0);

if (WIFEXITED(status)) {
    printf("Child exited with status: %d\n",
           WEXITSTATUS(status));
}

return 0;
```

OUTPUT:

```
sushmitha@LAPTOP-B907B06L:~$ nano skill8.c
sushmitha@LAPTOP-B907B06L:~$ gcc skill8.c -o skill8
sushmitha@LAPTOP-B907B06L:~$ ./skill8
myshell> pwd
/home/sushmitha
```